

Contracts vulnerabilities

Vulnerabilities list:

Contracts vulnerabilities	1
Vulnerabilities list:	1
Involved contracts and level of the bugs	2
Vulnerabilities	2
1. tokenURI function	2
2. create function	2
3. update function (zero bonds)	3
4. update function (replacing agent Ids)	4
5. drain function	4
6. _checkTokenStakingDeposit function	5
7. _isRatio function	5
8. stake function	6
9. unstake function	7
10. checkpoint function: O(n) complexity due to gas overflow	7
11. deploy function	8
12. create function	8
13. _claim function	9
14. execTransaction return value in RecoveryModule and other multisig creating contracts	10
15. registerAgentsWithSignature operator whitelist bypass	10
16. registerAgentsWithSignature missing msg.value validation	11
17. slash mechanism abuse by service owner	11
18. registerAgentsWithSignature missing deadline and maximum bond parameters	12
19. registerAgents agent instance registration DoS	13
20. slash and proportional RewardDistributionType split	13
21. checkpoint function during absence of rewards	14
22. _withdraw function reentrancy in StakingBase	15
23. Custom reward distributor staking contract address	16
24. Custom reward distributor code existence check	16
25. calculateStakingLastReward rounding dust	17

Involved contracts and level of the bugs

The present document aims to point out some vulnerabilities in the [autonolas-registry](#) contracts.

Vulnerabilities

1. tokenURI function

Severity: Low

The following function is implemented in the GenericRegistry contract:

```
function tokenURI(uint256 unitId) public view virtual override
returns (string memory)
```

This function is defined by the [EIP-721 standard](#). The standard states that the function is supposed to throw if **unitId** is not a valid NFT. However, in our contract, the function does not revert if the **unitId** is out of bounds, but just returns the value of a string with the defined prefix and 64 zeros derived from a zero bytes32 value.

Therefore, we recommend checking the return value of this view function, and if the last 64 symbols are zero, consider it to be an invalid NFT. Also one might use the **exists()** function to preliminary check if the requested NFT Id exists.

2. create function

Severity: Low

The following function is implemented in the GnosisSafeMultisig contract:

```
function create(address[] memory owners, uint256 threshold, bytes
memory data) external returns (address multisig)
```

This function creates a Safe service multisig when the service is deployed. Since Autonolas protocol follows an optimistic design, none of the fields for the Safe multisig creation are restricted. This way, the service owner might pass the *payload* field as they feel fit for the purposes of the service multisig. That said, any possible malicious behavior can also be embedded in the *payload* value.

In the event of the intended malicious multisig creation, the Autonolas protocol is not affected, however, accounts interacting with the corresponding service might bear eventual consequences of such a setup.

We strongly recommend not abusing the *payload* field of the service multisig when deploying the service to perform any malicious actions.

3. `update` function (zero bonds)

Severity: Low

The following function is implemented in the ServiceRegistry and ServiceRegistryL2 contracts:

```
function update(address serviceOwner, bytes32 configHash, uint32[]  
memory agentIds, uint32 threshold, uint256 serviceId) external  
returns (bool success)
```

This function allows updating a service in a *pre-registration* state in a CRUD way. E.g. if there is a need to remove `agentIds[i]` from the canonical agents making up the service, then it is sufficient to call this function and update it in such a way that a corresponding slots field is set to zero, i.e., `agentParam[i].slots=0`, also adjusting the `threshold`.

When an agent slot is non-zero, and an operator can register an agent instance for that slot, it is necessary that the corresponding agent bond is non-zero. In the current implementation, there is no check for agent bonds to be different from zero if the corresponding agent slot is non-zero. This vulnerability would enable an operator to register an agent instance without the corresponding security bond. Hence, the operator would not be affected by any possible slashing condition if the total operator bond is equal to zero.

This vulnerability is addressed for the ServiceRegistry contract and ServiceRegistryL2 by adding the zero-value check on the service manager level. Specifically, [serviceManager](#) contract handles the [check](#) before calling the original serviceRegistry's `update()` method. See

<https://github.com/valory-xyz/autonolas-registries/blob/main/test/ServiceManagerToken.js#L326-L333C25> for a test proving that the issue is resolved.

In absence of redeploying a new manager for the ServiceRegistryL2 contract on other chains, we recommend that service owners assign a zero-value to agent bonds only if the corresponding agent slot is zero.

4. `update` function (replacing agent ids)

Severity: Low

The following function is implemented in the `ServiceRegistry` and `ServiceRegistryL2` contract:

```
function update(address serviceOwner, bytes32 configHash, uint32[]  
memory agentIds, uint32 threshold, uint256 serviceId) external  
returns (bool success)
```

As described earlier, this function allows updating a service in a *pre-registration* state in a CRUD way. However, considering that there is no possible direct damage to the protocol and to save on transaction gas costs, the function is implemented via an optimistic approach.

Specifically, the service owner might not specify that some of the *agent ids* of the previous setup must be taken out of the system (by setting corresponding *slots* variable to zero). This means that operators are able to register agent instances specifying non-declared service agent ids (as those were deliberately left in the corresponding map from the previous setup). This might lead to deploying the service on *agent ids* from the previous setup, declaring that they actually run on current ones (as retrieved via the `getService()` view function).

We strongly recommend not abusing the `update()` function in order to deploy the service to perform any malicious actions by using undeclared *agent ids*, since this behavior is easily spotted off-chain.

5. `drain` function

Severity: Informative

The following function is implemented in the `ServiceRegistryTokenUtility` contract:

```
function drain(address token) external returns (uint256 amount)
```

The primary purpose of this function is to allow the removal of slashed tokens, other than chain-native tokens, from the contract.

By design, in the current setup of the Treasury contract, there is currently no mechanism in place to facilitate the removal of tokens other than ETH that have not been added to the

Treasury through the treasury `depositTokenForOLAS()` method. Therefore, we strongly advise against assigning the drainer role to the Treasury contract for ServiceRegistryTokenUtility contract deployed on Ethereum.

6. `_checkTokenStakingDeposit` function

Severity: Informative

The following function is implemented in the ServiceRegistryTokenUtility contract:

```
function _checkTokenStakingDeposit(uint256 serviceId, uint256
stakingDeposit, uint32[] memory) internal view virtual
```

The primary purpose of this function is to ensure that the service owner's security deposit and the operator bonds are correctly configured. Specifically, it checks that the service owner's security deposit (*securityDeposit*) and the *bond* for each operator are greater than or equal to *minStakingDeposit*. Given that *securityDeposit* is defined as the maximum among the operator bonds ($\max_{bond}\{bond\}$), when *minStakingDeposit* equals *securityDeposit*, the following relationship holds:

$$minStakingDeposit = securityDeposit \geq bond \geq minStakingDeposit$$

This ensures that *securityDeposit* = *minStakingDeposit* = *bond* for each operator bond. It's important to note that the service registry and service registry utility tokens do not enforce this requirement at the service level.

If one attempts to stake a service with a *securityDeposit* equal to *minStakingDeposit* and operator bonds that differ (e.g., $bond[i] > bond[i]$), it is recommended to terminate and update the service configuration to ensure compatibility with the staking logic.

7. `_isRatio` function

Severity: Informative

The following function is implemented in the StakingActivityChecker contract:

```
function isRatioPass(uint256[] memory curNonces, uint256[] memory
lastNonces, uint256 ts)
```

This function checks if the service multisig liveness ratio meets the defined threshold. The provided implementation serves as an illustrative example, and we highlight that multisig nonces are not tamper-resistant (cf. [InternalAudit4](#) for more details on this). It is therefore recommended to extend the basic **isRatioPass()** functionality in the StakingActivityChecker to verify whether specific on-chain actions occur within designated time frames. For a tamper-resistant check on on-chain activity, you can consider the one implemented in MechActivityChecker.sol in this [repository](#).

Additionally, the protocol optimistically assumes that the StakingActivityChecker contract used for deploying staking instances is implemented with a correct logic. Therefore, unless unexpected behavior such as reverts or non-boolean returns occur, the contract's results will be considered accurate. However, this optimistic assumption can be exploited by malicious users. For instance, malicious users could deploy multiple contracts with flawed activity checks that always return true. They could then vote for these contracts, causing the OLAS amount to be distributed to all stakers, including those without activity. Conversely, malicious users could deploy contracts with incorrect liveness checks that always return false, leading to a situation where the OLAS amount is sent, but funds remain stuck in the staking contracts and cannot be recovered.

The following measures can be considered to mitigate eventual abuses:

1. Set a Sensible Threshold: The DAO needs to establish a sensible threshold to enable staking emissions.
2. On-Chain Blacklist: Implement an on-chain blacklist that can be updated through governance votes, allowing the community to monitor and exclude malicious contracts.
3. Off-Chain Reputation System: Consider using an off-chain reputation system, possibly leveraging oracles, to assess the trustworthiness of contracts.

8. stake function

Severity: Informative

The following function is implemented in the StakingFactory contract:

```
function stake(uint256 serviceId) external
```

The function stakes a specified service. However, if the service was evicted, it cannot be staked again until it is explicitly unstaked.

9. `unstake` function

Severity: Informative

The following function is implemented in the StakingBase contract:

```
function unstake(uint256 serviceId) external returns (uint256 reward)
```

The function unstakes a previously staked service. If there are no available rewards left on a staking contract, the service can be unstaked immediately at any time. However, if there are even small funds deposited on the staking contract, the service will not be unstaked. It is not considered to be a griefing attack, since the unstake time is pre-defined via a `minStakingDuration` parameter. After that time, the service is unstaked without any concern of zero or non-zero available rewards. When the service is staked, it implicitly agrees to be staked for at least the `minStakingDuration`.

10. `checkpoint` function: $O(n)$ complexity and DoS due to gas overflow

Severity: High

The following function is implemented in the StakingBase contract:

```
function checkpoint() external returns (uint256[] memory, uint256[] memory, uint256[] memory, uint256[] memory)
```

The function goes through all currently staked services and checks for their activity KPIs. As designed, the function is $O(n)$ in its complexity, where n is the number of staked services. The actual staking limit is 500 services per staking contract. With the recent Fusaka EVM upgrade being adopted, the hard cap on transaction gas limit is imposed. Governance action to limit staking number of slots takes roughly a week. During this time, if a staking contract is misconfigured and has more than 100 slots, there is a risk that having more number of services staked results in a failure of a checkpoint transaction. This scenario ultimately leads to all services being staked indefinitely. In time of waiting for the proposal properly limiting the number of staking contract services, launchers of staking contracts are urged to configure them with no more than 100 of staking slots.

11. `deploy` function

Severity: Informational

The following function is implemented in the `ServiceRegistry` and `ServiceRegistryL2` contracts:

```
function deploy(address serviceOwner, uint256 serviceId, address
multisigImplementation, bytes memory data) external returns (address
multisig)
```

This function is responsible for deploying a service by creating a multisig instance controlled by the set of service agent instances. When the deployment uses a `RecoveryModule`-enabled multisig implementation, an additional module is installed to allow the service multisig to recover ownership in the event that agent instance keys are accidentally lost.

While this recovery mechanism allows re-obtaining control over the service multisig at the protocol level, it does not guarantee recoverability for all downstream integrations. In particular, certain interactions may still rely on the original multisig owner keys rather than the recovered ownership state.

A notable example is the `PolySafe` (Gnosis Safe L2) integration used by Polymarket. In this context, if the original multisig owner key is lost, there is no supported mechanism to regain effective control over the multisig from Polymarket's perspective, even if ownership is recovered on-chain via the recovery module. As a consequence, critical operations such as:

- buying or selling conditional tokens
- future earning Polymarket liquidity rewards

may become permanently inaccessible.

Therefore, although the recovery module provides resilience against agent key loss at the service level, it does not fully mitigate the risk of loss of functionality for external systems that bind permissions to the original multisig owner keys.

12. `create` function

Severity: Low

The following function is implemented in the PolySafeCreatorWithRecoveryModule contract:

```
function create(address[] memory owners, uint256 threshold, bytes memory data) external returns (address multisig)
```

This function creates PolySafe multisig when deploying the service. Since the `data` payload in this function consists of signatures, there is a possibility to front-run this transaction, and the original transaction will fail.

There is no financial benefit of such an attack. However, if this unlikely event takes place and in order not to set up a separate service, users are advised to engage with the GnosisSafeSameAddressMultisig contract's `create()` function specific for PolySafes. This contract function call just sets the already created multisig (that was created by front running), and service deployment is successful.

13. `_claim` function

Severity: Low

The following function is implemented in the StakingBase contract:

```
function function _claim(uint256 serviceId, bool execCheckPoint) internal returns (uint256 reward)
```

This function claims accumulated rewards for a specified `serviceId`. It also supports calling a `checkpoint()` function before claiming rewards. However, the sequence of actions is not entirely correct. The service reward is obtained first, then a checkpoint is called, and verification for the non-zero reward is made on a value obtained before the checkpoint. This leads to scenarios where there was no reward before the checkpoint, and thus the function reverts.

In order to get a correct sequence of actions and avoid described scenarios, we recommend calling `checkpoint()` separately before calling the `claim()` function.

14. `execTransaction` return value in `RecoveryModule` and other multisig creating contracts

Severity: Low

Source: Code4rena 2026-01 Olas audit (submission #S-69)

The following issue affects the `RecoveryModule` and multisig creation contracts:

The following function is implemented `RecoveryModule` contract:

```
function recoverAccess(uint256 serviceId) external
```

The following function is implemented in the majority of multisig creating contracts in `contracts/multisigs` folder:

```
function create(address[] memory owners, uint256 threshold, bytes memory data) external returns (address multisig)
```

Those functions utilize `Safe's execTransaction` calls and must revert if they return false on failure. The current implementation ignores this return value, allowing recovery attempts or multisig creation to silently fail.

It is straightforward to perform the off-chain check for the recovery module. As for other multisig creating contracts invoked via the `create()` function during service deployment, the guard is enforced at the `ServiceManager` level (see PR #241).

15. `registerAgentsWithSignature` operator whitelist bypass

Severity: Low

Source: Code4rena 2026-01 Olas audit (submission #S-149)

The following function is implemented in the `ServiceManager` contract:

```
function registerAgentsWithSignature(address serviceOwner, uint256 serviceId, address[] memory agentInstances, uint32[] memory agentIds, uint256 deadline, bytes memory signature) external payable
```

This issue reports that non-whitelisted operators could potentially register agents via `registerAgentsWithSignature` when the service owner has an operator whitelist enabled. This bypass could allow unauthorized operators to bind agent instances to a service.

This is solely based on service owner responsibility and we don't expect any misbehavior against self. The service owner is the party that configures the whitelist and controls registration. An operator exploiting this would require the service owner to have provided or approved the signature in the first place.

16. `registerAgentsWithSignature` missing `msg.value` validation

Severity: Low

Source: Code4rena 2026-01 Olas audit (submission #S-1175)

The following function is implemented in the `ServiceManager` contract:

```
function registerAgentsWithSignature(address serviceOwner, uint256
serviceId, address[] memory agentInstances, uint32[] memory agentIds,
uint256 deadline, bytes memory signature) external payable
```

In `registerAgents()`, the native-token branch strictly enforces: `require(msg.value == agentInstances.length * BOND_WRAPPER)`. However, in `registerAgentsWithSignature()`, the function forwards `{value: agentInstances.length * BOND_WRAPPER}` to the registry without validating the caller-supplied `msg.value`.

If `msg.value` exceeds the required bond wrapper amount, the excess ETH is not refunded and remains permanently stored in the `ServiceManager` contract, as no refund or withdrawal mechanism exists. This does not enable theft or privilege escalation. However, it introduces a locked ETH risk, where users may unintentionally overpay and permanently lose funds.

This is not an exploitable vulnerability but represents improper input validation that can result in permanent ETH lock. The `registerAgentsWithSignature()` function needs to be corrected by adding the specified `msg.value` check. Note: One of the duplicate submissions (#S-1175) was assessed as out of scope per the Known Issues section regarding misconfigured registration from users.

17. `slash` mechanism abuse by service owner

Severity: Low

Source: Code4rena 2026-01 Olas audit (submission #S-430)

The following function is implemented in the ServiceRegistry and ServiceRegistryL2 contracts:

```
function slash(address[] memory agentInstances, uint96[] memory amounts, uint256 serviceId) external returns (bool success)
```

This issue affects the interaction between service owners, the multisig module system, and the ServiceRegistry slash mechanism.

The protocol is permissionless, and thus the decision to engage in a specific service is solely on the service owner and operators. There are several ways to install any number of modules while the service owner is the owner of the multisig. It cannot be defined which modules to check on. Verifying this off-chain is much easier and free of charge.

However, even if operators are slashed, the funds are locked on the ServiceRegistry contract itself, and could only be drained by the DAO. At that point the DAO will reimburse operators. This attack does not have any economical benefit for the service owner.

18. registerAgentsWithSignature missing deadline and maximum bond parameters

Severity: Low

Source: Code4rena 2026-01 Olas audit (submissions #S-858, #S-862)

The following function is implemented in the ServiceManager contract:

```
function registerAgentsWithSignature(address serviceOwner, uint256 serviceId, address[] memory agentInstances, uint32[] memory agentIds, uint256 deadline, bytes memory signature) external payable
```

The operator can always set token approval to zero, and thus even with a valid signature the register function is going to fail. However, there could be a scenario where an operator is required to register to another service owner and once again provides approval to the ServiceRegistryTokenUtility. Then, the previous service owner can utilize the non-expiring signature to front-run and perform registration and utilize the newly provided approval, despite the approval being zero previously.

As for the maximum bond signature, there are no deadlines for a signature provided by the operator to the service owner. If a user has a higher approval and/or different token approval in the future, a delayed registration via registerAgentsWithSignature could possibly affect operators.

19. `registerAgents` agent instance registration DoS

Severity: Low

Source: Code4rena 2026-01 Olas audit (submission #S-901)

This issue affects the agent instance registration logic in ServiceRegistry:

```
function registerAgents(address operator, uint256 serviceId,  
address[] memory agentInstances, uint32[] memory agentIds) external  
payable
```

An agent instance is just an address that can be swapped with an infinite number of others. However, this is a legitimate issue, because the DoS can be sustained at a very cheap rate considering service creation is permissionless. Although it is technically true that an agent instance can be repeatedly swapped, the same argument also applies where a malicious user can theoretically infinitely DoS an agent registration.

Medium severity was proposed but low is appropriate here considering this behavior is likely not incentivized due to gas costs. Ultimately this is not an issue for the protocol at all due to the unlimited number of addresses. Trying to control or whitelist all of them is something blockchain was not created for.

20. `slash` and `proportional` `RewardDistributionType` split

Severity: Informative

Source: Code4rena 2026-01 Olas audit (submission #S-885)

The following function is implemented in the ServiceRegistry and ServiceRegistryL2 contracts:

```
function slash(address[] memory agentInstances, uint96[] memory  
amounts, uint256 serviceId) external returns (bool success)
```

It allows service multisig to slash operator bonds for misbehaving agent instances.

The following function is implemented in the StakingBase contract:

```
function function _claim(uint256 serviceId, bool execCheckPoint)
internal returns (uint256 reward)
```

This function claims accumulated rewards for a specified `serviceId`. If proportional reward type is selected for the staking contract, each operator is assigned an equal portion of the reward, under the assumption that each has deposited the same bond amount.

However, some of the operator bonds could be slashed. It is reported that slashing must be taken into consideration. The mitigation step would include the change of `_getRewardReceiversAndAmounts()` function so that when the distribution type is Proportional, the function queries the actual bond amounts for each operator. Operators with reduced or zero bonds (due to slashing) should receive proportionally less or no rewards. This ensures accurate and fair distribution.

At the same time, slashing is done for the agent instance misbehaving, and slashed funds are withheld by the protocol, specifically by the `ServiceRegistry(L2)` contracts. If an agent instance continues to perform its staking routines - those need to be rewarded for accordingly. Thus it is not considered to be an issue for the protocol, and the fix is not required.

21. `checkpoint` function during absence of rewards

Severity: Informative

Source: Code4rena 2026-01 Olas audit (submission #S-763)

The following function is implemented in the `StakingBase` contract:

```
function checkpoint() external returns (uint256[] memory, uint256[]
memory, uint256[] memory, uint256[] memory)
```

The function goes through all currently staked services and checks for their activity KPIs. A time manipulation vulnerability allows inactive services to earn rewards for periods they provided no service. When `availableRewards` is zero, checkpoint operations are skipped, creating unmeasured time gaps that services can exploit by becoming active only when rewards become available again.

It is recommended to separate the check of activity from reward distribution by maintaining activity state regardless of reward availability. However, the protocol maintains staking rewards, and if cancelled, no such situation is possible. Also, these

actions could be prevented by ensuring at least wei levels of tokens within the staking contracts, which is easy to do considering reward deposits are permissionless.

22. `_withdraw` function reentrancy in StakingBase

Severity: Low

The following function is implemented in the StakingBase contract:

```
function _withdraw(uint256 serviceId, uint256 reward) internal
```

This internal function is called by `claim()`, `checkpointAndClaim()`, and `unstake()` to distribute rewards to receivers. It reads balance into a local variable, loops through receivers calling `_transfer()` for each, then writes balance back to storage after all transfers complete.

The developer comment at line 564 states: "reentrancy is not possible since reward is set to zero" — this is true for same-service reentrancy, since `mapServiceInfo[serviceId].reward` is zeroed before `_withdraw()` is called. Re-entering `claim()` for the same service would find zero reward and revert.

However, for `StakingNativeToken`, `_transfer()` sends ETH via `.call{value}` which gives the receiver a callback. A contract owning two services (A and B) on the same `StakingNativeToken` instance can exploit cross-service reentrancy:

1. `claim(serviceA) → _withdraw → _transfer` sends ETH → `receive()` callback
2. In callback: `claim(serviceB) → _withdraw → reads stale balance → writes balance`
3. Outer `_withdraw` writes its stale balance → overwrites the inner update

This desynchronizes the balance variable from actual contract holdings. For `StakingToken` (ERC20), `SafeTransferLib.safeTransfer` is used, which does not give the receiver a callback, so reentrancy is not possible.

Current deployment status:

- `StakingNativeToken` is not utilized on any of supported chains
- Only `StakingToken` (ERC20, no callback) contracts are in production

The severity is Low (latent) as the code bug is confirmed but not currently exploitable. It would become Medium/High if `StakingNativeToken` is actively used with multiple services having a contract as reward receiver.

We recommend adding a reentrancy guard (`_locked`) to `StakingBase` `claim()`, `unstake()`, `forcedUnstake()`, `checkpoint()`, and `checkpointAndClaim()`.

23. Custom reward distributor returning staking contract address

Severity: Low

The following function is implemented in the `StakingBase` contract:

```
function _getRewardReceiversAndAmounts(uint256 serviceId, uint256
reward) internal view returns (address[] memory receivers, uint256[]
memory amounts)
```

When `RewardDistributionType.Custom` is selected, this function calls the external distributor contract via `staticcall` to obtain reward receivers and amounts. The validation checks (line 720-733) verify that `sum(amounts) == reward` and that the arrays are non-empty, but do not check whether any receiver is `address(this)` (the staking contract itself).

If a staker sets a custom distributor that returns the staking contract as a reward receiver, `_withdraw()` transfers tokens to `address(this)`:

- `StakingToken`: ERC20 transfer to self succeeds, tokens stay in the contract but balance accounting desynchronizes (tracked balance < actual balance).
- `StakingNativeToken`: ETH sent to self triggers `receive()` which increments balance and `availableRewards`, then `_withdraw` overwrites balance with a stale value.

The impact is self-inflicted: the staker loses their own reward. Tokens become permanently orphaned in the contract as there is no drain or rescue function. Other stakers are not directly affected since their rewards are calculated from `availableRewards` which decrements correctly.

We recommend adding a check in the `_getRewardReceiversAndAmounts` Custom path to reject `address(this)` and `address(0)` as receivers.

24. Custom reward distributor code existence check

Severity: Low

The following function is implemented in the `StakingBase` contract:

```
function _stake(uint256 serviceId, address serviceOwner, uint256[]  
memory serviceAgentIds) internal
```

When `RewardDistributionType.Custom` is selected during staking, the function validates only that the distributor address is not `address(0)` (line 824-827). It does not check `code.length > 0`.

If an EOA or a not-yet-deployed contract is set as the custom distributor, the service's `claim()` will revert due to empty return data from the staticcall to the EOA. This effectively bricks the service's reward claims until `unstake()` or `forcedUnstake()` is called.

The impact is self-inflicted: the service owner bricks their own claims. Other stakers are unaffected since claim is per-service and not batched.

We recommend adding `require(customDistributorAddr.code.length > 0)` in the stake function when the Custom distribution type is used.

25. `calculateStakingLastReward` rounding dust

Severity: Informative

The following function is implemented in the `StakingBase` contract:

```
function calculateStakingLastReward(uint256 serviceId) external view  
returns (uint256 reward)
```

The `checkpoint()` function distributes rounding dust (up to `numServices - 1` wei) to the service at index 0 in the staked services set (line 1002-1004). However, the `calculateStakingLastReward()` view function (line 1146) does not account for this bonus when computing the expected reward.

The impact is cosmetic: the view function shows a slightly lower reward than what is actually distributed during the checkpoint. The discrepancy is at most `numServices - 1` wei, which is negligible in practice.

No fix is required, but callers of `calculateStakingLastReward()` should be aware that the actual reward for the first staked service may be marginally higher than reported.